

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И
ОПТИКИ ФАКУЛЬТЕТ

ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №1 по курсу «ООП» Тема: Управление инвентарем
в ролевой игре.

Вариант 25 Выполнил: Чекушин В.С. К3241

Преподаватель: Николай Кочубеев

Санкт-Петербург 2025 г.

Оглавление

Структура проекта 3

Реализация базового класса и наследования 3

Паттерн Builder для создания предметов 3

Логика инвентаря и апгрейда..... 4

Unit-тестирование 4

Вывод 4

Структура проекта

Система управления инвентарем персонажа, реализующая механизмы хранения, улучшения и использования игровых предметов.

- **Базовая логика (Item):** Абстрактный класс, задающий общие свойства для всех предметов (ID, уровень, описание).
- **Типы предметов:**
 - Weapon: Оружие с параметром урона.
 - Armor: Броня с защитой и механикой сетов (комплектов).
 - Potion: Расходные материалы (зелья) с различными эффектами.
 - QuestItem: Специальные предметы, защищенные от удаления.
- **Паттерны:** Использование паттерна **Builder** для гибкого создания объектов брони.
- **Управление (Inventory):** Контейнер для предметов с бизнес-логикой апгрейда и расчета бонусов.

Реализация базового класса и наследования

Все предметы наследуются от Item, что позволяет хранить их в едином списке List<Item>.

```
public abstract class Item
{
    public string Id { get; } = Guid.NewGuid().ToString();
    public string Name { get; set; }
    public int Level { get; set; } = 1;

    protected Item(string name, string desc)
    {
        Name = name;
        Description = desc;
    }
}
```

Паттерн Builder для создания предметов

Класс ArmorBuilder позволяет конструировать сложные объекты брони, задавая только нужные параметры.

```
public class ArmorBuilder
{
    private string name = "Armor";
    private int def = 0;
    private ArmorSlot slot = ArmorSlot.Chest;
    private string? setId = null;

    public ArmorBuilder SetName(string n) { name = n; return this; }
    public ArmorBuilder SetDefense(int d) { def = d; return this; }
    public ArmorBuilder SetSlot(ArmorSlot s) { slot = s; return this; }
    public Armor Build() => new Armor(name, "", 1, def, slot, setId);
}
```

Логика инвентаря и апгрейда

Класс Inventory обрабатывает механику улучшения предметов, которая динамически изменяет характеристики в зависимости от типа объекта.

```
public void UpgradeItem(Item item)
{
    item.Level++;
    if (item is Weapon w) w.Damage += 2;
    if (item is Armor a) a.Defense += 2;
    if (item is Potion p) p.Power += 5;
}

public bool RemoveItem(string id)
{
    var item = Items.FirstOrDefault(i => i.Id == id);
    if (item == null || item is QuestItem) return false; // Защита квестовых предметов
    Items.Remove(item);
    return true;
}
```

Unit-тестирование

Тесты проверяют корректность расчета бонусов от комплектов брони и невозможность удаления квестовых предметов.

```
[Fact]
public void RemoveItem_Works_QuestItemNotRemoved()
{
    var inv = new Inventory();
    var quest = new QuestItem("Quest1", "Important");
    inv.AddItem(quest);

    var removed = inv.RemoveItem(quest.Id);

    Assert.False(removed);
    Assert.Contains(quest, inv.Items);
}
```

Вывод

- **Безопасность данных:** Квестовые предметы нельзя удалить из инвентаря благодаря проверке типа (is QuestItem).
- **Гибкость апгрейда:** Метод UpgradeItem использует приведение типов для специфического изменения характеристик (урон для оружия, защита для брони).
- **Система сетов:** Реализован расчет бонусов за ношение двух и более предметов из одного комплекта (SetId).